



Diff_ExtractorLLM

— 0 removals

343 lines

```

1 # main.py
2 # Purpose: PoC LLM extractor that reads
  your existing per-listing JSONL
  records,
3 # fetches the original TXT, asks an LLM
  (Vertex AI) to extract fields, and
  writes
4 # a sibling "<post_id>_llm.jsonl" to
  the NEW 'jsonl_llm/' sub-directory.
5 #
6 # FINAL FIXES INCLUDED:
7 # 1. Schema updated to use "type":
  "string" + "nullable": True.
8 # 2. system_instruction removed from
  GenerationConfig and merged into
  prompt.
9 # 3. LLM_MODEL set to 'gemini-2.5-
  flash' (Fixes 404/NotFound error).
10 # 4. "additionalProperties": False
  removed from schema (Fixes internal
  ParseError).
11 # 5. Non-breaking spaces (U+00A0)
  replaced with standard spaces (U+0020).
  <--- FIX FOR THIS ERROR
12
13 import os
14 import re
15 import json
16 import logging
17 import traceback
18 import time
19 from datetime import datetime, timezone
20
21 from flask import Request, jsonify
22 from google.api_core import retry as
  gax_retry
23 from google.cloud import storage
24
25 # ---- REQUIRED VERTEX AI IMPORTS ----

```

+ 5 additions

346 lines

```

1 # main.py
2 # Purpose: PoC LLM extractor that reads
  your existing per-listing JSONL
  records,
3 # fetches the original TXT, asks an LLM
  (Vertex AI) to extract fields, and
  writes
4 # a sibling "<post_id>_llm.jsonl" to
  the NEW 'jsonl_llm/' sub-directory.
5 #
6 # FINAL FIXES INCLUDED:
7 # 1. Schema updated to use "type":
  "string" + "nullable": True.
8 # 2. system_instruction removed from
  GenerationConfig and merged into
  prompt.
9 # 3. LLM_MODEL set to 'gemini-2.5-
  flash' (Fixes 404/NotFound error).
10 # 4. "additionalProperties": False
  removed from schema (Fixes internal
  ParseError).
11 # 5. Non-breaking spaces (U+00A0)
  replaced with standard spaces (U+0020).
  <--- FIX FOR THIS ERROR
12
13 import os
14 import re
15 import json
16 import logging
17 import traceback
18 import time
19 from datetime import datetime, timezone
20
21 from flask import Request, jsonify
22 from google.api_core import retry as
  gax_retry
23 from google.cloud import storage
24
25 # ---- REQUIRED VERTEX AI IMPORTS ----

```

```

26 import vertexai
27 from vertexai.generative_models import
    GenerativeModel, GenerationConfig,
    Content
28 from google.api_core.exceptions import
    ResourceExhausted, InternalServerError,
    Aborted, DeadlineExceeded
29
30 # ----- ENV -----
    -----
31 PROJECT_ID          =
    os.getenv("PROJECT_ID", "")
32 REGION              =
    os.getenv("REGION", "us-central1")
33 BUCKET_NAME         =
    os.getenv("GCS_BUCKET", "")
34 STRUCTURED_PREFIX   =
    os.getenv("STRUCTURED_PREFIX",
    "structured")
35 LLM_PROVIDER        =
    os.getenv("LLM_PROVIDER",
    "vertex").lower()
36 LLM_MODEL            =
    os.getenv("LLM_MODEL", "gemini-2.5-
    flash")
37 OVERWRITE_DEFAULT   =
    os.getenv("OVERWRITE", "false").lower()
    == "true"
38 MAX_FILES_DEFAULT   =
    int(os.getenv("MAX_FILES", "0") or 0)
39
40 # GCS READ RETRY - Use default
    transient error logic
41 READ_RETRY = gax_retry.Retry(
42
    predicate=gax_retry.if_transient_error,
43     initial=1.0, maximum=10.0,
    multiplier=2.0, deadline=120.0
44 )
45
46 # LLM API RETRY PREDICATE
47 def _if_llm_retryable(exception):
48     """Checks if the exception is
    transient and should trigger a
    retry."""
49     return isinstance(exception,
    (ResourceExhausted,
    InternalServerError, Aborted,
    DeadlineExceeded))

```

```

26 import vertexai
27 from vertexai.generative_models import
    GenerativeModel, GenerationConfig,
    Content
28 from google.api_core.exceptions import
    ResourceExhausted, InternalServerError,
    Aborted, DeadlineExceeded
29
30 # ----- ENV -----
    -----
31 PROJECT_ID          =
    os.getenv("PROJECT_ID", "")
32 REGION              =
    os.getenv("REGION", "us-central1")
33 BUCKET_NAME         =
    os.getenv("GCS_BUCKET", "")
34 STRUCTURED_PREFIX   =
    os.getenv("STRUCTURED_PREFIX",
    "structured")
35 LLM_PROVIDER        =
    os.getenv("LLM_PROVIDER",
    "vertex").lower()
36 LLM_MODEL            =
    os.getenv("LLM_MODEL", "gemini-2.5-
    flash")
37 OVERWRITE_DEFAULT   =
    os.getenv("OVERWRITE", "false").lower()
    == "true"
38 MAX_FILES_DEFAULT   =
    int(os.getenv("MAX_FILES", "0") or 0)
39
40 # GCS READ RETRY - Use default
    transient error logic
41 READ_RETRY = gax_retry.Retry(
42
    predicate=gax_retry.if_transient_error,
43     initial=1.0, maximum=10.0,
    multiplier=2.0, deadline=120.0
44 )
45
46 # LLM API RETRY PREDICATE
47 def _if_llm_retryable(exception):
48     """Checks if the exception is
    transient and should trigger a
    retry."""
49     return isinstance(exception,
    (ResourceExhausted,
    InternalServerError, Aborted,
    DeadlineExceeded))

```

```

50
51 # LLM CALL RETRY
52 LLM_RETRY = gax_retry.Retry(
53     predicate=_if_llm_retryable,
54     initial=5.0, maximum=30.0,
55     multiplier=2.0, deadline=180.0,
56 )
57 storage_client = storage.Client()
58 _CACHE_MODEL_OBJ = None
59
60 # Accept BOTH run id styles:
61 RUN_ID_ISO_RE =
62     re.compile(r"^\d{8}T\d{6}Z$")
63 RUN_ID_PLAIN_RE =
64     re.compile(r"^\d{14}$")
65
66 # ----- HELPERS -----
67
68 def _get_vertex_model() ->
69     GenerativeModel:
70     """Initializes and returns the
71     cached Vertex AI model object (thread-
72     safe for CF)."""
73     global _CACHE_MODEL_OBJ
74     if _CACHE_MODEL_OBJ is None:
75         if not PROJECT_ID:
76             raise
77         RuntimeError("PROJECT_ID environment
78         variable is missing.")
79
80     # Initialize client once per
81     container lifecycle
82
83     vertexai.init(project=PROJECT_ID,
84         location=REGION)
85     _CACHE_MODEL_OBJ =
86     GenerativeModel(LLM_MODEL)
87     logging.info(f"Initialized
88     Vertex AI model: {LLM_MODEL} in
89     {REGION}")
90     return _CACHE_MODEL_OBJ
91
92 def _list_structured_run_ids(bucket:
93     str, structured_prefix: str) ->
94     list[str]:
95     """

```

```

50
51 # LLM CALL RETRY
52 LLM_RETRY = gax_retry.Retry(
53     predicate=_if_llm_retryable,
54     initial=5.0, maximum=30.0,
55     multiplier=2.0, deadline=180.0,
56 )
57 storage_client = storage.Client()
58 _CACHE_MODEL_OBJ = None
59
60 # Accept BOTH run id styles:
61 RUN_ID_ISO_RE =
62     re.compile(r"^\d{8}T\d{6}Z$")
63 RUN_ID_PLAIN_RE =
64     re.compile(r"^\d{14}$")
65
66 # ----- HELPERS -----
67
68 def _get_vertex_model() ->
69     GenerativeModel:
70     """Initializes and returns the
71     cached Vertex AI model object (thread-
72     safe for CF)."""
73     global _CACHE_MODEL_OBJ
74     if _CACHE_MODEL_OBJ is None:
75         if not PROJECT_ID:
76             raise
77         RuntimeError("PROJECT_ID environment
78         variable is missing.")
79
80     # Initialize client once per
81     container lifecycle
82
83     vertexai.init(project=PROJECT_ID,
84         location=REGION)
85     _CACHE_MODEL_OBJ =
86     GenerativeModel(LLM_MODEL)
87     logging.info(f"Initialized
88     Vertex AI model: {LLM_MODEL} in
89     {REGION}")
90     return _CACHE_MODEL_OBJ
91
92 def _list_structured_run_ids(bucket:
93     str, structured_prefix: str) ->
94     list[str]:
95     """

```

```

82     List 'structured/run_id=*/'
    directories and return normalized
    run_ids.
83     """
84     it =
    storage_client.list_blobs(bucket,
    prefix=f"{structured_prefix}/",
    delimiter="/")
85     for _ in it:
86         pass
87
88     runs = []
89     for pref in getattr(it, "prefixes",
    []):
90         tail =
    pref.rstrip("/").split("/")[1]
91         if tail.startswith("run_id="):
92             cand =
    tail.split("run_id=", 1)[1]
93             if
    RUN_ID_ISO_RE.match(cand) or
    RUN_ID_PLAIN_RE.match(cand):
94                 runs.append(cand)
95     return sorted(runs)
96
97
98 def _normalize_run_id_iso(run_id: str)
    -> str:
99     """
100     Normalize run_id to ISO8601 Z
    string for provenance.
101     """
102     try:
103         if RUN_ID_ISO_RE.match(run_id):
104             dt =
    datetime.strptime(run_id,
    "%Y%m%dT%H%M%SZ").replace(tzinfo=timezo
    ne.utc)
105         elif
    RUN_ID_PLAIN_RE.match(run_id):
106             dt =
    datetime.strptime(run_id,
    "%Y%m%d%H%M%S").replace(tzinfo=timezone
    .utc)
107         else:
108             raise
    ValueError("unsupported run_id")
109     return
    dt.isoformat().replace("+00:00", "Z")

```

```

82     List 'structured/run_id=*/'
    directories and return normalized
    run_ids.
83     """
84     it =
    storage_client.list_blobs(bucket,
    prefix=f"{structured_prefix}/",
    delimiter="/")
85     for _ in it:
86         pass
87
88     runs = []
89     for pref in getattr(it, "prefixes",
    []):
90         tail =
    pref.rstrip("/").split("/")[1]
91         if tail.startswith("run_id="):
92             cand =
    tail.split("run_id=", 1)[1]
93             if
    RUN_ID_ISO_RE.match(cand) or
    RUN_ID_PLAIN_RE.match(cand):
94                 runs.append(cand)
95     return sorted(runs)
96
97
98 def _normalize_run_id_iso(run_id: str)
    -> str:
99     """
100     Normalize run_id to ISO8601 Z
    string for provenance.
101     """
102     try:
103         if RUN_ID_ISO_RE.match(run_id):
104             dt =
    datetime.strptime(run_id,
    "%Y%m%dT%H%M%SZ").replace(tzinfo=timezo
    ne.utc)
105         elif
    RUN_ID_PLAIN_RE.match(run_id):
106             dt =
    datetime.strptime(run_id,
    "%Y%m%d%H%M%S").replace(tzinfo=timezone
    .utc)
107         else:
108             raise
    ValueError("unsupported run_id")
109     return
    dt.isoformat().replace("+00:00", "Z")

```

```

110     except Exception:
111         return
        datetime.now(timezone.utc).isoformat().
        replace("+00:00", "Z")
112
113
114 def
    _list_per_listing_jsonl_for_run(bucket:
    str, run_id: str) -> list[str]:
115     """
116     Return *input* per-listing JSONL
    object names for a given run_id
117     (assumes inputs are in 'jsonl/').
118     """
119     prefix = f"
    {STRUCTURED_PREFIX}/run_id=
    {run_id}/jsonl/"
120     bucket_obj =
    storage_client.bucket(bucket)
121     names = []
122     for b in
    bucket_obj.list_blobs(prefix=prefix):
123         if not
    b.name.endswith(".jsonl"):
124             continue
125             names.append(b.name)
126     return names
127
128
129 def _download_text(blob_name: str) ->
    str:
130     bucket =
    storage_client.bucket(BUCKET_NAME)
131     blob = bucket.blob(blob_name)
132     return
    blob.download_as_text(retry=READ_RETRY,
    timeout=120)
133
134
135 def _upload_jsonl_line(blob_name: str,
    record: dict):
136     bucket =
    storage_client.bucket(BUCKET_NAME)
137     blob = bucket.blob(blob_name)
138     line = json.dumps(record,
    ensure_ascii=False, separators=(",",
    ":", " ")) + "\n"
139     blob.upload_from_string(line,
    content_type="application/x-ndjson")

```

```

110     except Exception:
111         return
        datetime.now(timezone.utc).isoformat().
        replace("+00:00", "Z")
112
113
114 def
    _list_per_listing_jsonl_for_run(bucket:
    str, run_id: str) -> list[str]:
115     """
116     Return *input* per-listing JSONL
    object names for a given run_id
117     (assumes inputs are in 'jsonl/').
118     """
119     prefix = f"
    {STRUCTURED_PREFIX}/run_id=
    {run_id}/jsonl/"
120     bucket_obj =
    storage_client.bucket(bucket)
121     names = []
122     for b in
    bucket_obj.list_blobs(prefix=prefix):
123         if not
    b.name.endswith(".jsonl"):
124             continue
125             names.append(b.name)
126     return names
127
128
129 def _download_text(blob_name: str) ->
    str:
130     bucket =
    storage_client.bucket(BUCKET_NAME)
131     blob = bucket.blob(blob_name)
132     return
    blob.download_as_text(retry=READ_RETRY,
    timeout=120)
133
134
135 def _upload_jsonl_line(blob_name: str,
    record: dict):
136     bucket =
    storage_client.bucket(BUCKET_NAME)
137     blob = bucket.blob(blob_name)
138     line = json.dumps(record,
    ensure_ascii=False, separators=(",",
    ":", " ")) + "\n"
139     blob.upload_from_string(line,
    content_type="application/x-ndjson")

```

```

140
141
142 def _blob_exists(blob_name: str) ->
    bool:
143     bucket =
        storage_client.bucket(BUCKET_NAME)
144     return
        bucket.blob(blob_name).exists()
145
146
147 def _safe_int(x):
148     try:
149         if x is None or x == "":
150             return None
151         return int(str(x).replace(", ",
            "").strip()))
152     except Exception:
153         return None
154
155
156 # ----- VERTEX AI CALL -
    -----
157 def _vertex_extract_fields(raw_text:
    str) -> dict:
158     """
159     Ask Gemini to return JSON with
    exactly: price, year, make, model,
    mileage.
160     """
161     model = _get_vertex_model()
162
163     # Strict JSON schema - FIX: Removed
    "additionalProperties": False
164     schema = {
165         "type": "object",
166         "properties": {
167             "price": {"type":
    "integer", "nullable": True},
168             "year": {"type": "integer",
    "nullable": True},
169             "make": {"type": "string",
    "nullable": True},
170             "model": {"type": "string",
    "nullable": True},

```

```

140
141
142 def _blob_exists(blob_name: str) ->
    bool:
143     bucket =
        storage_client.bucket(BUCKET_NAME)
144     return
        bucket.blob(blob_name).exists()
145
146
147 def _safe_int(x):
148     try:
149         if x is None or x == "":
150             return None
151         return int(str(x).replace(", ",
            "").strip()))
152     except Exception:
153         return None
154
155
156 # ----- VERTEX AI CALL -
    -----
157 def _vertex_extract_fields(raw_text:
    str) -> dict:
158     """
159     Ask Gemini to return JSON with
    exactly: price, year, make, model,
    transmission, mileage.
160     """
161     model = _get_vertex_model()
162
163     # Strict JSON schema - FIX: Removed
    "additionalProperties": False
164     schema = {
165         "type": "object",
166         "properties": {
167             "price": {"type":
    "integer", "nullable": True},
168             "year": {"type": "integer",
    "nullable": True},
169             "make": {"type": "string",
    "nullable": True},
170             "model": {"type": "string",
    "nullable": True},
171             "transmission": {"type":
    "string", "nullable": True},

```

```

171         "mileage": {"type":
"integer", "nullable": True},
172     },
173     "required": ["price", "year",
"make", "model", "mileage"]
174 }
175
176 # System instruction (will be
prepended to the prompt)
177 sys_instr = (
178     "Extract ONLY the following
fields from the input text. "
179     "Return a strict JSON object
that conforms to the provided schema. "
180     "If a value is not present, use
null. "
181     "Rules: integers for
price/year/mileage; price in USD;
mileage in miles; "
182     "do not infer values not
explicitly present; do not add extra
keys."
183 )
184
185 # FIX: Combine instruction and text
into one prompt string (SDK
compatibility)
186 prompt = f"
{sys_instr}\n\nTEXT:\n{raw_text}"
187
188 gen_cfg = GenerationConfig(
189     # FIX: system_instruction
removed to fix TypeError
190     temperature=0.0,
191     top_p=1.0,
192     top_k=40,
193     candidate_count=1,
194
    response_mime_type="application/json",
195     response_schema=schema,
196 )
197
198 # --- LLM CALL WITH RETRY ---

```

```

172         "mileage": {"type":
"integer", "nullable": True},
173     },
174     "required": ["price", "year",
"make", "model", "transmission",
"mileage"]
175 }
176
177 # System instruction (will be
prepended to the prompt)
178 sys_instr = (
179     "Extract ONLY the following
fields from the input text. "
180     "Return a strict JSON object
that conforms to the provided schema. "
181     "If a value is not present, use
null. "
182     "Rules: integers for
price/year/mileage; price in USD;
mileage in miles; "
183     "the transmission can be manual
or automatic, or if not listed, write
null."
184     "do not infer values not
explicitly present; do not add extra
keys."
185 )
186
187 # FIX: Combine instruction and text
into one prompt string (SDK
compatibility)
188 prompt = f"
{sys_instr}\n\nTEXT:\n{raw_text}"
189
190 gen_cfg = GenerationConfig(
191     # FIX: system_instruction
removed to fix TypeError
192     temperature=0.0,
193     top_p=1.0,
194     top_k=40,
195     candidate_count=1,
196
    response_mime_type="application/json",
197     response_schema=schema,
198 )
199
200 # --- LLM CALL WITH RETRY ---

```

```

199     max_attempts = 3
200     resp = None
201     for attempt in range(max_attempts):
202         try:
203             # Pass the single string
                prompt
204             resp =
                model.generate_content(prompt,
                generation_config=gen_cfg)
205             break
206         except Exception as e:
207             # Includes the 404/NotFound
                error from the previous run
208             if not _if_llm_retryable(e)
                or attempt == max_attempts - 1:
209
                logging.error(f"Fatal/non-retryable LLM
                error or max retries reached: {e}")
210                 raise
211
212                 sleep_time =
                LLM_RETRY._calculate_sleep(attempt)
213                 logging.warning(f"Transient
                LLM error on attempt
                {attempt+1}/{max_attempts}. Retrying in
                {sleep_time:.2f}s...")
214                 time.sleep(sleep_time)
215
216             if resp is None:
217                 raise RuntimeError("LLM call
                failed after all retries.")
218
219             parsed = json.loads(resp.text)
220
221             # Normalize fields post-extraction
222             parsed["price"] =
                _safe_int(parsed.get("price"))
223             parsed["year"] =
                _safe_int(parsed.get("year"))
224             parsed["mileage"] =
                _safe_int(parsed.get("mileage"))
225
226             def _norm_str(s):
227                 if s is None: return None
228                 s = str(s).strip()
229                 return s if s else None
230
231             parsed["make"] =
                _norm_str(parsed.get("make"))

```

```

201     max_attempts = 3
202     resp = None
203     for attempt in range(max_attempts):
204         try:
205             # Pass the single string
                prompt
206             resp =
                model.generate_content(prompt,
                generation_config=gen_cfg)
207             break
208         except Exception as e:
209             # Includes the 404/NotFound
                error from the previous run
210             if not _if_llm_retryable(e)
                or attempt == max_attempts - 1:
211
                logging.error(f"Fatal/non-retryable LLM
                error or max retries reached: {e}")
212                 raise
213
214                 sleep_time =
                LLM_RETRY._calculate_sleep(attempt)
215                 logging.warning(f"Transient
                LLM error on attempt
                {attempt+1}/{max_attempts}. Retrying in
                {sleep_time:.2f}s...")
216                 time.sleep(sleep_time)
217
218             if resp is None:
219                 raise RuntimeError("LLM call
                failed after all retries.")
220
221             parsed = json.loads(resp.text)
222
223             # Normalize fields post-extraction
224             parsed["price"] =
                _safe_int(parsed.get("price"))
225             parsed["year"] =
                _safe_int(parsed.get("year"))
226             parsed["mileage"] =
                _safe_int(parsed.get("mileage"))
227
228             def _norm_str(s):
229                 if s is None: return None
230                 s = str(s).strip()
231                 return s if s else None
232
233             parsed["make"] =
                _norm_str(parsed.get("make"))

```



```

232     parsed["model"] =
        _norm_str(parsed.get("model"))
233
234     return parsed
235
236
237 # ----- HTTP ENTRY -----
        -----
238 def llm_extract_http(request: Request):
239     """
240     Reads latest (or requested) run's
        per-listing JSONL inputs and writes LLM
        outputs.
241     """
242
        logging.getLogger().setLevel(logging.INFO)
243
244     if not BUCKET_NAME:
245         return jsonify({"ok": False,
            "error": "missing GCS_BUCKET env"}),
            500
246
247     if not PROJECT_ID:
248         return jsonify({"ok": False,
            "error": "missing PROJECT_ID env"}),
            500
249
250     if LLM_PROVIDER != "vertex":
251         return jsonify({"ok": False,
            "error": "PoC supports
            LLM_PROVIDER='vertex' only"}), 400
252
253     # Body overrides
254     try:
255         body =
            request.get_json(silent=True) or {}
256     except Exception:
257         body = {}
258
259     run_id = body.get("run_id")
260     max_files =
        int(body.get("max_files") or
        MAX_FILES_DEFAULT or 0)
261     overwrite =
        bool(body.get("overwrite")) if
        "overwrite" in body else
        OVERWRITE_DEFAULT
262
263     # Pick newest run if not provided
264     if not run_id:

```

```

234     parsed["model"] =
        _norm_str(parsed.get("model"))
235
236     return parsed
237
238
239 # ----- HTTP ENTRY -----
        -----
240 def llm_extract_http(request: Request):
241     """
242     Reads latest (or requested) run's
        per-listing JSONL inputs and writes LLM
        outputs.
243     """
244
        logging.getLogger().setLevel(logging.INFO)
245
246     if not BUCKET_NAME:
247         return jsonify({"ok": False,
            "error": "missing GCS_BUCKET env"}),
            500
248
249     if not PROJECT_ID:
250         return jsonify({"ok": False,
            "error": "missing PROJECT_ID env"}),
            500
251
252     if LLM_PROVIDER != "vertex":
253         return jsonify({"ok": False,
            "error": "PoC supports
            LLM_PROVIDER='vertex' only"}), 400
254
255     # Body overrides
256     try:
257         body =
            request.get_json(silent=True) or {}
258     except Exception:
259         body = {}
260
261     run_id = body.get("run_id")
262     max_files =
        int(body.get("max_files") or
        MAX_FILES_DEFAULT or 0)
263     overwrite =
        bool(body.get("overwrite")) if
        "overwrite" in body else
        OVERWRITE_DEFAULT
264
265     # Pick newest run if not provided
266     if not run_id:

```

```

263         runs =
            _list_structured_run_ids(BUCKET_NAME,
            STRUCTURED_PREFIX)
264         if not runs:
265             return jsonify({"ok":
False, "error": f"no run_ids found
under {STRUCTURED_PREFIX}/"}, 200
266             run_id = runs[-1]
267
268         structured_iso =
            _normalize_run_id_iso(run_id)
269
270         inputs =
            _list_per_listing_jsonl_for_run(BUCKET_
            NAME, run_id)
271         if not inputs:
272             return jsonify({"ok": True,
"run_id": run_id, "processed": 0,
"written": 0, "skipped": 0, "errors":
0}), 200
273         if max_files > 0:
274             inputs = inputs[:max_files]
275
276         logging.info(f"Starting LLM
extraction for run_id={run_id}
({len(inputs)} files to process)")
277
278         processed = written = skipped =
errors = 0
279
280         for in_key in inputs:
281             processed += 1
282             try:
283                 # Read the tiny JSON line
(single record)
284                 raw_line =
                    _download_text(in_key).strip()
285                 if not raw_line:
286                     raise ValueError("empty
input jsonl")
287                 base_rec =
                    json.loads(raw_line)
288
289                 post_id =
                    base_rec.get("post_id")
290                 if not post_id:
291                     raise
ValueError("missing post_id in input
record")

```

```

265         runs =
            _list_structured_run_ids(BUCKET_NAME,
            STRUCTURED_PREFIX)
266         if not runs:
267             return jsonify({"ok":
False, "error": f"no run_ids found
under {STRUCTURED_PREFIX}/"}, 200
268             run_id = runs[-1]
269
270         structured_iso =
            _normalize_run_id_iso(run_id)
271
272         inputs =
            _list_per_listing_jsonl_for_run(BUCKET_
            NAME, run_id)
273         if not inputs:
274             return jsonify({"ok": True,
"run_id": run_id, "processed": 0,
"written": 0, "skipped": 0, "errors":
0}), 200
275         if max_files > 0:
276             inputs = inputs[:max_files]
277
278         logging.info(f"Starting LLM
extraction for run_id={run_id}
({len(inputs)} files to process)")
279
280         processed = written = skipped =
errors = 0
281
282         for in_key in inputs:
283             processed += 1
284             try:
285                 # Read the tiny JSON line
(single record)
286                 raw_line =
                    _download_text(in_key).strip()
287                 if not raw_line:
288                     raise ValueError("empty
input jsonl")
289                 base_rec =
                    json.loads(raw_line)
290
291                 post_id =
                    base_rec.get("post_id")
292                 if not post_id:
293                     raise
ValueError("missing post_id in input
record")

```

```

292
293         source_txt_key =
base_rec.get("source_txt")
294         if not source_txt_key:
295             raise
ValueError("missing source_txt in input
record")
296
297         # Output path: uses
'jsonl_llm/' folder
298         out_prefix =
in_key.rsplit("/", 2)[0] + "/jsonl_llm"
299         out_key = out_prefix +
f"/{post_id}_llm.jsonl"
300
301         if not overwrite and
_blob_exists(out_key):
302             skipped += 1
303             continue
304
305         # Fetch the raw listing
TXT; send to LLM
306         raw_listing =
_download_text(source_txt_key)
307
308         parsed =
_vertex_extract_fields(raw_listing)
309
310         # Compose final record
311         out_record = {
312             "post_id": post_id,
313             "run_id":
base_rec.get("run_id", run_id),
314             "scraped_at":
base_rec.get("scraped_at",
structured_iso),
315             "source_txt":
source_txt_key,
316             "price":
parsed.get("price"),
317             "year":
parsed.get("year"),
318             "make":
parsed.get("make"),
319             "model":
parsed.get("model"),
320             "mileage":
parsed.get("mileage"),

```

```

294
295         source_txt_key =
base_rec.get("source_txt")
296         if not source_txt_key:
297             raise
ValueError("missing source_txt in input
record")
298
299         # Output path: uses
'jsonl_llm/' folder
300         out_prefix =
in_key.rsplit("/", 2)[0] + "/jsonl_llm"
301         out_key = out_prefix +
f"/{post_id}_llm.jsonl"
302
303         if not overwrite and
_blob_exists(out_key):
304             skipped += 1
305             continue
306
307         # Fetch the raw listing
TXT; send to LLM
308         raw_listing =
_download_text(source_txt_key)
309
310         parsed =
_vertex_extract_fields(raw_listing)
311
312         # Compose final record
313         out_record = {
314             "post_id": post_id,
315             "run_id":
base_rec.get("run_id", run_id),
316             "scraped_at":
base_rec.get("scraped_at",
structured_iso),
317             "source_txt":
source_txt_key,
318             "price":
parsed.get("price"),
319             "year":
parsed.get("year"),
320             "make":
parsed.get("make"),
321             "model":
parsed.get("model"),
322             "mileage":
parsed.get("mileage"),

```

```

321         "llm_provider":
"vertex",
322         "llm_model": LLM_MODEL,
323         "llm_ts":
datetime.now(timezone.utc).isoformat().
replace("+00:00", "Z"),
324     }
325
326     _upload_jsonl_line(out_key,
out_record)
327     written += 1
328
329     except Exception as e:
330         errors += 1
331         logging.error(f"LLM
extraction failed for {in_key}:
{e}\n{traceback.format_exc()}")
332
333     result = {
334         "ok": True,
335         "version": "extractor-llm-poc",
336         "run_id": run_id,
337         "processed": processed,
338         "written": written,
339         "skipped": skipped,
340         "errors": errors,
341     }
342     logging.info(json.dumps(result))
343     return jsonify(result), 200

```

```

323         "transmission":
parsed.get("transmission"),
324         "llm_provider":
"vertex",
325         "llm_model": LLM_MODEL,
326         "llm_ts":
datetime.now(timezone.utc).isoformat().
replace("+00:00", "Z"),
327     }
328
329     _upload_jsonl_line(out_key,
out_record)
330     written += 1
331
332     except Exception as e:
333         errors += 1
334         logging.error(f"LLM
extraction failed for {in_key}:
{e}\n{traceback.format_exc()}")
335
336     result = {
337         "ok": True,
338         "version": "extractor-llm-poc",
339         "run_id": run_id,
340         "processed": processed,
341         "written": written,
342         "skipped": skipped,
343         "errors": errors,
344     }
345     logging.info(json.dumps(result))
346     return jsonify(result), 200

```